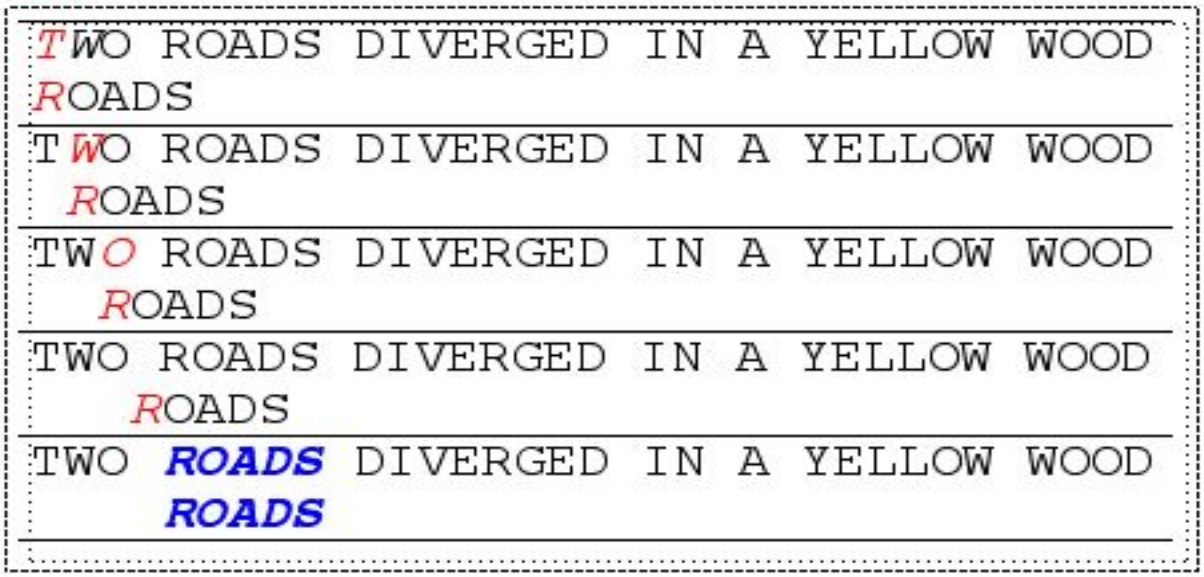
Strings and Pattern Matching

Brute Force

- The Brute Force algorithm compares the pattern to the text, one character at a time, until unmatching characters are found



Compared characters are *italicized*.

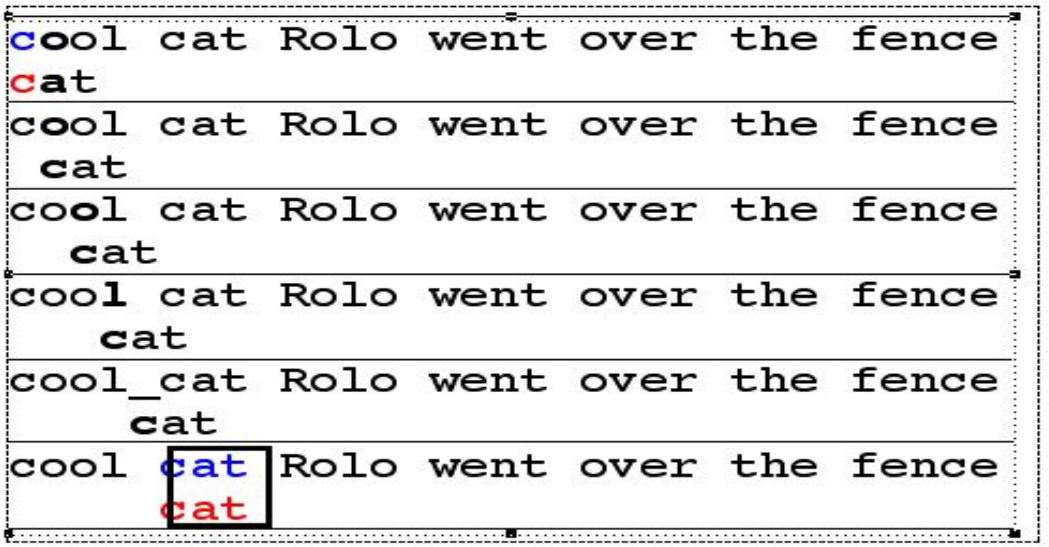
Correct matches are in **boldface** type.

- The algorithm can be designed to stop on either the first occurrence of the pattern, or upon reaching the end of the text.

Brute Force Pseudo-Code

- Here's the pseudo-code

do if (text letter == pattern letter)
compare next letter of pattern to next letter of text
else move pattern down text by one letter
while (entire pattern found or end of text)



Brute Force-Complexity

- Given a pattern M characters in length, and a text N characters in length...
- Worst case: compares pattern to each substring of text of length M. For example, M=5.
- This kind of case can occur for image data.

Total number of comparisons: M (N-M+1)

Worst case time complexity: O(MN)

Brute Force-Complexity(cont.)

- Given a pattern M characters in length, and a text N characters in length...
- Best case if pattern found: Finds pattern in first M positions of text. For example, M=5.

1) AAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAA **5 comparisons made**

Total number of comparisons: M
Best case time complexity: O(M)

Brute Force-Complexity(cont.)

- Given a pattern M characters in length, and a text N characters in length...
- Best case if pattern not found: Always mismatch on first character. For example, M=5.

Total number of comparisons: N

Best case time complexity: O(N)

Rabin-Karp

- The Rabin-Karp string searching algorithm calculates a **hash value** for the pattern, and for each M-character subsequence of text to be compared.
- If the hash values are unequal, the algorithm will calculate the hash value for next M-character sequence.
- If the hash values are equal, the algorithm will do a Brute Force comparison between the pattern and the M-character sequence.
- In this way, there is only one comparison per text subsequence, and Brute Force is only needed when hash values match.

Rabin-Karp Example

- Hash value of “AAAAA” is 37
- Hash value of “AAAAH” is 100

1) AAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

2) AAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

3) AAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

4) AAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

...

N) AAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
5 comparisons made 100=100

Rabin-Karp Algorithm

pattern is M characters long
hash_p=hash value of pattern
hash_t=hash value of first M letters in body of text
do
    if (hash_p == hash_t)
     brute force comparison of pattern and selected section of text
          hash_t= hash value of next section of text, one character over
while (end of text)

Rabin-Karp

- Common Rabin-Karp questions:

  "What is the hash function used to calculate values for character sequences?"

  "Isn't it time consuming to hash very one of the M-character sequences in the text body?"

- To answer some of these questions, we'll have to get mathematical.

Hash Function

- Let b be the number of letters in the alphabet. The text subsequence t[i .. i+M-1] is mapped to the number

$$x(i) = t[i] \cdot b^{M-1} + t[i+1] \cdot b^{M-2} + \ldots + t[i+M-1]$$

- Furthermore, given x(i) we can compute x(i+1) for the next subsequence t[i+1 .. i+M] in constant time, as follows:

$$x(i+1) = t[i+1] \cdot b^{M-1} + t[i+2] \cdot b^{M-2} + \ldots + t[i+M]$$

$$x(i+1) = x(i) \cdot b$$ Shift left one digit

$$- t[i] \cdot b^{M}$$ Subtract leftmost digit

$$+ t[i+M]$$ Add new rightmost digit

- In this way, we never explicitly compute a new value. We simply adjust the existing value as we move over one character.

Rabin-Karp Math Example

- Let's say that our alphabet consists of 10 letters.
- our alphabet = a, b, c, d, e, f, g, h, i, j
- Let's say that "a" corresponds to 1, "b" corresponds to 2 and so on.

The hash value for string "cah" would be ...

3*100 + 1*10 + 8*1 = 318

Rabin-Karp Mods

- If M is large, then the resulting value ($\sim b^M$) will be enormous. For this reason, we hash the value by taking it **mod** a prime number **q**.
- The **mod** function is particularly useful in this case due to several of its inherent properties:

  $[(x \bmod q) + (y \bmod q)] \bmod q = (x+y) \bmod q$

  $(x \bmod q) \bmod q = x \bmod q$
- For these reasons:

$h(i)=((t[i]\ b^{M-1} \bmod q) + (t[i+1]\ b^{M-2} \bmod q) + \ldots + (t[i+M-1] \bmod q)) \bmod q$

$h(i+1) = (h(i)\ b \bmod q$

Shift left one digit

$-t[i]\ b^M \bmod q$

Subtract leftmost digit

$+t[i+M] \bmod q)$

Add new rightmost digit

$\bmod q$

Rabin-Karp Complexity

- If a sufficiently large prime number is used for the *hash function*, the hashed values of two different patterns will usually be distinct.
- If this is the case, searching takes O(N) time, where N is the number of characters in the larger body of text.
- It is always possible to construct a scenario with a worst case complexity of O(MN). This, however, is likely to happen only if the prime number used for hashing is small.